

ForeXplore 项目介绍

1. 项目定位

ForeXplore 是一个面向软件模块复用的开源工作流框架。它把已有代码仓库中的实现，与目标软件系统中的 `class` 或 `function` 建立关联，帮助开发者完成以下过程：

1. 从目标工程的模块树中选择待实现或待增强的符号。
2. 使用自然语言描述功能需求、行为约束和接口要求。
3. 从一个或多个多语言代码仓库中检索相似实现。
4. 对召回结果进行过滤和重排，向用户呈现 Top-K 候选方案及其依据。
5. 由用户选择候选、指定复用策略并补充迁移约束。
6. 将候选实现翻译、包装或桥接到目标模块。
7. 验证接口契约、代码质量和工程可用性。
8. 以可审阅、可追踪的补丁事务回填目标工程。

ForeXplore 关注的不是孤立地生成一段代码，而是构建一条可解释、可干预、可验证的代码复用链路。人始终保留候选方案和最终补丁的决定权。

2. 核心 workflow

目标模块树

|

v

选择 `class` / `function` + 描述需求与约束

|

v

多路召回：语义检索 / 符号检索 / 结构检索

|

v

过滤与重排：相关性 / 接口契合度 / 依赖与风险

|

v

Top-K 候选展示 + 人工选择与备注

|

v

翻译 / 桥接 / 包装 / 同语言复用

|

v

接口映射 + 补丁生成

|

v

静态检查 / 契约检查 / 测试验证

|

v

补丁预览 + 人工确认

|

v

事务回填 + 检查点记录

这条链路被拆分为稳定的 workflow 状态和可替换的能力端口。检索引擎、模型、向量数据库以及 IDE 宿主都可以替换，而不改变用户完成任务的基本流程。

3. 检索与重排

3.1 检索输入

检索不是只接收一段自然语言。一个完整请求同时包含目标符号的结构化上下文：

输入	作用
目标符号	指定待实现的 <code>class</code> 或 <code>function</code>
符号签名	提供参数、返回值和异步等接口信息
目标语言	判断直接复用或跨语言迁移成本
自然语言需求	描述业务行为、异常处理、性能和并发要求
Top-K	控制最终呈现给用户的方案数量
检索模式	选择语义、结构或混合检索
仓库范围	限定允许检索的代码库与数据边界

当前契约由 `SearchRequest` 表达，检索实现通过 `CodeSearchPort` 注入。这样可以先使用 Mock 或简单检索器验证交互流程，再逐步接入真实代码索引和模型服务。

3.2 多路召回

生产检索服务计划从大规模、多语言代码仓中召回一个大于 Top-K 的初始候选集。不同召回通道负责发现不同类型的相似性：

- **语义召回**：根据需求描述、注释、文档和代码语义查找行为相似的实现。
- **符号召回**：根据类名、函数名、签名、参数和返回类型查找接口相近的实现。
- **结构召回**：利用语法树、调用关系、依赖关系和目录结构查找实现形态相近的模块。
- **文本召回**：使用关键词和精确匹配补充可解释、低成本的候选来源。

实际部署可以采用混合策略：结构化索引负责缩小搜索空间，文本与向量检索负责召回，LLM 只读取少量高价值候选并理解其实现逻辑。这样能够控制大仓库下的上下文长度、索引成本和推理开销。

3.3 候选过滤

召回结果在重排前需要经过基础过滤，避免把明显不可用的代码交给后续模型：

- 确认候选粒度为可迁移的 `class` 或 `function`。
- 检查仓库来源、许可证和代码版本是否可追踪。
- 排除缺失主体代码、依赖无法解析或接口信息不完整的候选。
- 根据用户配置限制语言、仓库、依赖类型和安全边界。
- 合并重复实现或同一实现的多个近似片段。

3.4 重排与 Top-K

重排阶段不只判断“代码看起来是否相似”，还要判断它能否以合理成本进入目标模块。当前候选契约包含以下评分维度：

- **semantic**：候选行为与自然语言需求的语义匹配度。
- **symbol**：类名、函数名、签名和符号结构的匹配度。
- **contract**：参数、返回值、错误语义和调用方式的契合度。
- **overall**：综合得分，用于生成最终候选顺序。

后续真实重排器还可以将许可证、依赖复杂度、跨语言成本、测试覆盖和安全风险作为过滤条件或排序特征。评分必须同时返回证据，不能只返回一个不可解释的数值。

最终 Top-K 方案应向用户展示：

- 来源仓库、文件路径、许可证和语言。
- 符号签名、实现摘要和核心代码预览。
- 各维度评分以及匹配理由。
- 可直接保留的行为和接口。
- 外部依赖、迁移成本与潜在风险。

用户可以比较候选并选择其中一个，也可以返回需求阶段调整约束后重新检索。候选选择不是自动提交动作，而是翻译与适配阶段的明确入口。

4. 人工决策

检索结果提供事实和建议，最终复用策略由用户决定。选择候选后，用户还可以补充例如“保持现有公开签名”“禁止新增全局状态”“依赖必须通过构造函数注入”等约束。

这些信息会与目标符号、原始需求和候选实现一起组成 **AdaptationRequest**，因此人工备注是后续生成与验证的正式输入，而不是只显示在界面上的临时文本。

ForeXplore 支持四种适配策略：

策略	适用场景
translate	将跨语言实现翻译为目标工程语言，并遵循目标工程惯例
bridge	保留源语言实现，通过 RPC、FFI、进程或服务边界调用
wrap	使用适配器包装候选接口，使其满足目标模块契约
reuse	同语言且契约接近时直接复用，并进行必要的小范围修改

5. 翻译与接口适配

翻译不是逐行替换语法，而是把候选实现的行为迁移到目标模块的接口和运行环境中。适配服务应按以下步骤处理：

1. **提取源行为**：识别主流程、状态、错误路径、并发语义和依赖。

2. **读取目标契约**：分析目标签名、类型、调用方、工程规范和可用依赖。
3. **制定适配计划**：确定翻译、桥接、包装或直接复用策略。
4. **建立接口映射**：逐项处理参数、返回值、错误、依赖和生命周期。
5. **生成目标代码**：生成满足目标语言和模块边界的实现。
6. **生成文件补丁**：把代码组织为可定位、可审阅的文件变更，而非直接写入工作区。

接口映射使用结构化数据记录，并支持以下动作：

- **rename**：符号或字段改名。
- **convert**：类型、数据结构或错误模型转换。
- **inject**：注入目标工程所需依赖。
- **preserve**：保留目标模块现有契约或候选行为。

例如，将 Python 异步缓存实现迁移到 TypeScript 时，适配阶段不仅需要翻译 **async** 逻辑，还要明确 Python 异常如何映射到目标错误类型、锁和 single-flight 语义如何实现、缓存存储如何注入，以及公开的 **Promise<Quote>** 返回契约是否保持不变。

6. 验证

生成代码在回填前必须经过验证。验证结果与补丁一起展示，用户能够判断哪些检查已经通过、哪些问题仍需处理。

验证可以分为五层：

1. **语法验证**：目标代码能够被解析和格式化。
2. **类型与构建验证**：目标工程能够通过类型检查或编译。
3. **接口契约验证**：公开签名、输入输出、错误语义和调用方式符合目标模块要求。
4. **行为验证**：相关单元测试、集成测试或生成的针对性测试能够通过。
5. **工程验证**：依赖、许可证、安全策略和修改范围符合工程约束。

验证结果至少区分 **pass** 与 **warn**。生产实现还应保存具体命令、输出摘要和对应文件，保证结果可以复查。关键验证失败时不应自动回填，而应回到适配阶段修正或交由用户决定。

7. 补丁预览与事务回填

适配服务返回 **FilePatch[]**，其中包含文件路径、创建或修改状态、增删行统计和具体 diff。GUI 在真正写入代码前展示：

- 所有待修改或新增的文件。
- 每个文件的增删内容。
- 接口映射结果。
- 自动验证结果和剩余风险。

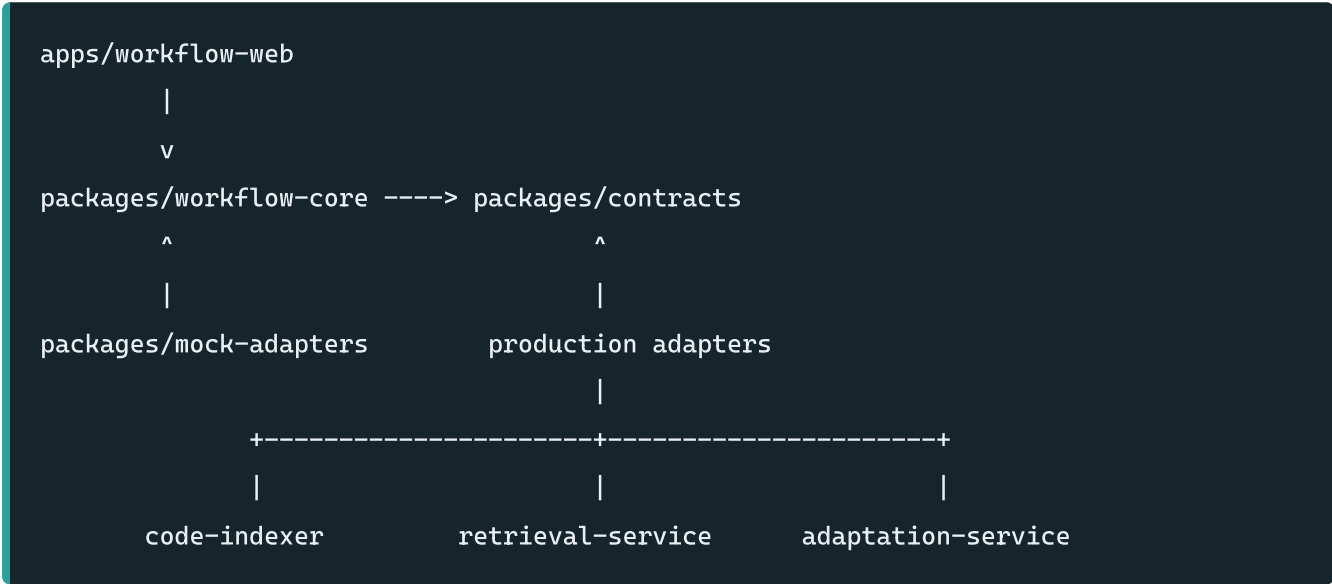
- 本次适配采用的策略。

用户确认后， `CodeBackfillPort` 才创建工作区编辑事务。成功回填会返回：

- 实际写入的文件列表。
- 本次事务的检查点 ID。

检查点用于审计、比较和后续回滚。真实 IDE 集成中可以将回填映射为 VS Code `WorkspaceEdit`、独立 Git 分支或其他具备原子性和恢复能力的编辑机制。

8. 架构边界



各模块职责如下：

模块	职责
<code>apps/workflow-web</code>	模块选择、需求输入、候选对比、补丁审阅等 GUI 交互
<code>packages/contracts</code>	检索、候选、适配、补丁和模块符号的共享类型
<code>packages/workflow-core</code>	workflow 状态、状态转换和能力端口
<code>packages/mock-adapters</code>	用于演示完整链路的静态数据和 Mock 实现
<code>services/code-indexer</code>	仓库发现、语言解析、符号和依赖索引
<code>services/retrieval-service</code>	多路召回、过滤、重排与 Top-K 返回
<code>services/adaptation-service</code>	翻译、桥接、接口映射、补丁生成和验证

核心 workflow 不依赖某个具体 IDE。VS Code 插件、Trae 二次开发或自建 Agent 都可以作为宿主，通过相同端口连接模块树、检索服务和工作区编辑能力。

9. 当前实现状态

当前仓库已经完成：

- React/Vite 工作流 GUI。
- `class` / `function` 粒度的静态模块树。
- 目标选择、需求输入、Top-K 候选对比和人工备注交互。
- 工作流状态机以及检索、适配、回填端口。
- 翻译、桥接、包装和直接复用策略的数据契约。
- 接口映射、验证结果和文件 diff 的展示。
- 从模块选择到 Mock 回填的端到端测试。

当前仍属于 Mock 或预留边界的部分包括：

- 真实工程的符号解析和增量索引。
- 面向大规模多语言代码仓的召回与重排服务。
- 基于模型或规则的代码翻译与桥接生成。
- 编译、测试、安全与许可证验证执行器。
- 对真实工作区的事务写入和恢复机制。
- VS Code、Trae 或其他 IDE 的正式集成。

10. 设计原则

- **人在回路中**：候选选择和最终回填必须经过明确确认。
- **证据优先**：检索结果同时提供来源、评分依据、兼容点和风险。
- **契约优先**：翻译围绕目标模块接口进行，而不是追求源代码逐行等价。
- **先验证后写入**：任何生成结果先形成补丁并验证，再进入目标工程。
- **过程可追踪**：候选来源、人工备注、适配策略、验证结果和回填检查点都应保留。
- **实现可替换**：索引、检索、模型、IDE 和存储通过稳定端口接入。
- **渐进式落地**：先用 Mock 跑通交互和接口，再逐步替换为生产服务。

ForeXplore 最终希望让跨仓库、跨语言的代码复用从一次不可控的生成行为，变成一条能够比较、选择、验证和追踪的软件工程 workflow。